



Relatório Técnico – Projeto de Introdução à Linguagem de Programação / unidade 2 Calculadora Multifuncional

Aluno: Bruno Ferreira da Silva

Introdução

Este projeto representa a evolução de uma calculadora multifuncional desenvolvida em linguagem C como parte das atividades acadêmicas da disciplina. O trabalho demonstra a aplicação prática dos conceitos teóricos abordados na segunda unidade, com foco especial na transição de estruturas estáticas para dinâmicas e no uso eficiente de recursos de memória.

Os conceitos da U2 foram aplicados de forma integrada em todo o projeto, permitindo a criação de um sistema mais robusto e eficiente. A calculadora científica agora oferece um sistema de histórico expansível, enquanto o conversor de unidades apresenta uma arquitetura mais flexível para adição de novas categorias. A organização do código foi planejada para facilitar a manutenção e futuras expansões, com clara separação entre as funcionalidades principais e tratamento adequado de recursos.

Os principais desafios técnicos enfrentados incluíram o gerenciamento seguro de ponteiros e a implementação de algoritmos eficientes para manipulação de strings, especialmente na formatação de resultados numéricos. A superação desses

desafios proporcionou um aprendizado significativo sobre o controle de recursos em linguagem C.

Metodologia

O desenvolvimento seguiu uma abordagem incremental, utilizando o compilador GCC em ambiente Windows com suporte do MinGW. A estratégia adotada priorizou a refatoração gradual do código existente da U1, incorporando progressivamente os novos conceitos da U2 enquanto mantinha a funcionalidade central intacta.

A organização do código foi estruturada em módulos funcionais independentes, onde cada componente principal possui responsabilidades bem definidas. Essa abordagem facilitou significativamente a manutenção e permitiu o desenvolvimento paralelo de diferentes funcionalidades. A separação entre a calculadora científica e o conversor de unidades demonstrou ser particularmente eficaz, pois isolou as complexidades específicas de cada módulo.

A metodologia de implementação das estruturas de dados complexas seguiu princípios de design modular, com matrizes sendo utilizadas para gerenciar os menus de unidades de forma eficiente. A estratégia de desenvolvimento iterativo permitiu testar cada novo conceito isoladamente antes de integrá-lo ao sistema principal, reduzindo a complexidade do debugging.

Análise do Código

3.1 Strings e Manipulação de Texto

As strings foram utilizadas extensivamente para criar interfaces de usuário dinâmicas e para armazenar expressões matemáticas completas. A implementação de funções dedicadas à manipulação de texto permitiu a formatação inteligente de números, removendo zeros desnecessários e melhorando a legibilidade dos resultados. As operações de concatenação foram essenciais para construir expressões matemáticas complexas durante a execução de cálculos sequenciais.

A manipulação de strings também foi fundamental na geração de menus interativos, onde arrays de strings armazenavam as opções disponíveis para o usuário. Essa abordagem demonstrou como a gestão eficiente de texto pode melhorar significativamente a experiência do usuário final.

3.2 Estruturas de Repetição Aninhadas

As estruturas de repetição aninhadas foram aplicadas principalmente na interface da calculadora científica, criando dois níveis de interação: um loop externo para sessões de cálculo múltiplas e um loop interno para operações matemáticas sequenciais dentro de cada sessão. Essa arquitetura permitiu que os usuários realizassem múltiplos cálculos sem precisar reiniciar o programa, melhorando consideravelmente a usabilidade.

A complexidade dos algoritmos implementados com estruturas aninhadas foi mantida linear em relação ao número de operações, garantindo bom desempenho mesmo em sessões prolongadas. Os casos de uso mais complexos envolveram a validação de entrada do usuário em múltiplos níveis, onde as estruturas aninhadas provaram ser essenciais para criar uma experiência interativa fluida.

3.3 Matrizes e Estruturas de Dados

As matrizes foram implementadas como arrays bidimensionais de strings para gerenciar os sistemas de menus do conversor de unidades. Essa estratégia permitiu associar cada categoria de conversão com suas respectivas unidades de medida de forma organizada e expansível. As operações suportadas incluíram acesso indexado eficiente e percorrimento sequencial para exibição das opções disponíveis.

A estratégia de percorrimento adotada utilizou loops simples para iterar através das matrizes, apresentando ao usuário interfaces claras e consistentes. Essa implementação demonstrou como estruturas de dados aparentemente simples podem resolver problemas complexos de organização de informação quando aplicadas de forma adequada.

3.4 Ponteiros e Eficiência

Os ponteiros foram utilizados para melhorar a eficiência do sistema em múltiplos aspectos. Na passagem de parâmetros, o uso de ponteiros evitou a cópia desnecessária de grandes estruturas de dados, reduzindo o consumo de memória e melhorando o desempenho. A implementação de arrays dinâmicos para o histórico de cálculos dependeu fundamentalmente do uso correto de ponteiros para gerenciar o crescimento da memória alocada.

Um dos usos mais sofisticados de ponteiros foi na implementação de arrays de ponteiros para funções, permitindo chamadas dinâmicas aos diferentes conversores de unidades baseadas na seleção do usuário. Essa abordagem trouxe flexibilidade ao sistema e facilitou a adição de novos tipos de conversão no futuro.

3.5 Alocação Dinâmica e Gerenciamento de Memória

A alocação dinâmica foi necessária principalmente para implementar o sistema de histórico de cálculos expansível. A estratégia de gerenciamento de memória adotada começava com uma alocação inicial moderada e expandia a capacidade conforme necessário através de realocação, sempre verificando o sucesso de cada operação de alocação.

Para garantir que não ocorressem vazamentos de memória, foi implementado um sistema rigoroso de liberação de recursos que incluía verificações de nulidade antes de cada operação de liberação e um mecanismo de limpeza final no término do programa. As vantagens da alocação dinâmica para este projeto foram evidentes na flexibilidade para lidar com quantidades variáveis de dados e na eficiência no uso da memória, alocando apenas o estritamente necessário.

Dificuldades e Soluções

O desenvolvimento do projeto apresentou diversos desafios técnicos que exigiram soluções criativas e aprofundamento nos conceitos da U2. A principal dificuldade esteve relacionada ao gerenciamento de ponteiros, especialmente na coordenação entre diferentes partes do código que compartilhavam acesso às mesmas regiões de memória. A solução envolveu a implementação de protocolos

claros para inicialização, uso e liberação de ponteiros, com verificações sistemáticas de validade antes de cada acesso.

Outro desafio significativo foi a implementação do sistema de redimensionamento dinâmico do histórico de cálculos. A estratégia de duplicação progressiva da capacidade mostrou-se eficiente, mas exigiu cuidado especial no tratamento de falhas de alocação para garantir que o programa se comportasse graciosamente mesmo em situações de memória insuficiente.

As dificuldades com manipulação de strings concentraram-se na formatação de números, particularmente na remoção de zeros decimais desnecessários sem afetar a precisão dos cálculos. A solução final envolveu a criação de uma função especializada que analisava a string resultante e removia seletivamente os caracteres desnecessários, preservando a integridade do valor numérico.

Conclusão

A evolução do projeto da U1 para a U2 demonstrou claramente as vantagens dos conceitos avançados de programação em C. A transição de estruturas estáticas para dinâmicas trouxe benefícios tangíveis em termos de flexibilidade e eficiência, permitindo que o sistema se adaptasse melhor às necessidades do usuário. Foi possível perceber que o uso de ponteiros com alocação dinâmica torna o código muito mais otimizado e rápido, especialmente no manejo de grandes volumes de dados.

Entre as possíveis melhorias futuras, destaca-se a necessidade de melhorar a sequência de operações da calculadora científica para suportar expressões mais complexas e implementar precedência de operadores. A arquitetura atual já oferece uma base sólida para essas expansões, graças à organização modular adotada.

Para a U3, os próximos passos naturais incluem separar o código em arquivos diferentes, deixando-o mais organizado e fácil de entender, além de implementar persistência de dados através de arquivos para manter o histórico entre execuções do programa. O aprendizado obtido com os conceitos da U2 proporcionou as ferramentas necessárias para enfrentar esses desafios futuros com confiança.